دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش آزمایش ۶

طراحی یک انکوباتور

Digital Systems Design Laboratory

اعضای گروه

پوریا عرفانزاده ۴۰۱۱۱۰۹۱۸
فاطیما تیمارچی ۴۰۲۱۰۵۸۰۲
آیه صابری ۴۰۲۱۰۶۱۴۵

نام درس

آزمایشگاه طراحی سیستم‌های دیجیتال

استاد درس

دکتر علیرضا اجلالی

سال تحصیلی ۱۴۰۴–۱۴۰۵

۱.۶ مقدمه و اهداف

هدف این آزمایش، طراحی و پیاده‌سازی واحد کنترل دیجیتال یک انکوباتور[۱] است. مطابق دستورکار، در این سامانه یک حسگر دما، یک واحد گرم‌کننده و یک واحد خنک‌کنندهٔ مجهّز به پنکه وجود دارد. حسگر، دمای محفظه را که میان -10 تا $+60$ درجهٔ سانتی‌گراد متغیّر است می‌خواند و در قالب یک عدد هشت‌بیتی به واحد کنترل تحویل می‌دهد. واحد کنترل باید بر پایهٔ این دما دو تصمیم بگیرد: نخست آنکه گرم‌کننده و خنک‌کننده را چگونه فعّال و غیرفعّال کند، و دوم آنکه در صورت فعّال بودن خنک‌کننده، دور پنکهٔ آن را چگونه تنظیم نماید.

رفتار خواسته‌شده در دستورکار با دو نمودار حالت توصیف شده است و بنابراین طرح، طبیعتاً به دو ماشین حالت متناهی[۲] می‌انجامد که یکی وضعیّت گرمایش و سرمایش و دیگری دور پنکه را اداره می‌کند. دستورکار همچنین تصریح می‌کند که حسگر، گرم‌کننده و خنک‌کننده وجود خارجی ندارند و باید دما را خود به مدار اعمال کرد و واکنش سامانه را با چراغ‌ها نمایش داد.

روند انجام آزمایش

ابتدا دو نمودار حالت دستورکار به‌دقّت استخراج و به‌صورت جدول درآمد تا هیچ یال و هیچ آستانه‌ای از قلم نیفتد. سپس واحد کنترل به‌صورت رفتاری و در قالب دو ماشین حالت مور[۳] توصیف شد. برای ارزیابی، دو میز آزمون[۴] مکمّل ساخته شد: یکی شبیه‌سازی حلقه‌بسته[۵] با مدل فرایند[۶] که رفتار واقعی محفظه را تقلید می‌کند، و دیگری آزمون هدفمند مرزها که درستی تک‌تک آستانه‌های دمایی را می‌سنجد. پس از اجرای هر دو در ModelSim، طرح در Quartus II سنتز شد و افزون بر گزارش منابع و تحلیل زمانی، تأثیر دو شیوهٔ رمزگذاری حالت[۷] بر مساحت و بیشینه‌بسامد نیز مقایسه شد.

[1] Incubator
[2] Finite State Machine
[3] Moore Machine
[4] Testbench
[5] Closed-Loop Simulation
[6] Plant Model
[7] State Encoding

۲.۶ مشخّصات خواسته‌شده و دو نمودار حالت

الف) ماشین حالت اصلی

ماشین نخست، گرم‌کننده و خنک‌کننده را اداره می‌کند و سه حالت دارد. جدول زیر حالت‌ها و یال‌های آن را دقیقاً مطابق نمودار دستورکار گردآوری می‌کند.

حالت	گرم‌کننده	خنک‌کننده	یال‌های خروجی
S1	خاموش	خاموش	به S2 اگر $T > 35$؛ به S3 اگر $T < 15$
S2	خاموش	روشن	به S1 اگر $T < 25$
S3	روشن	خاموش	به S1 اگر $T > 30$

جدول ۱: حالت‌ها و گذارهای ماشین حالت اصلی؛ دماها بر حسب درجه‌ی سانتی‌گراد

ب) ماشین حالت دور پنکه

ماشین دوم، دور پنکهٔ خنک‌کننده را تنظیم می‌کند. سه حالت کاری و یک حالت غیرفعّال دارد.

حالت	دور پنکه	یال‌های خروجی
OUT	—	به S1 هنگام فعّال شدن ماشین اصلی در حالت S2
S1	4 دور بر ثانیه[۸]	به S2 اگر $T > 40$
S2	6 دور بر ثانیه	به S3 اگر $T > 45$؛ به S1 اگر $T < 35$
S3	8 دور بر ثانیه	به S2 اگر $T < 40$

جدول ۲: حالت‌ها و گذارهای ماشین حالت دور پنکه

ج) قاعده‌ی پیوند دو ماشین

دستورکار یک قید مهم را در متن روشن کرده است: نمودار حالت دوم تنها هنگامی از حالت OUT خود خارج می‌شود که نمودار نخست در حالت S2 باشد، یعنی خنک‌کننده روشن باشد؛ و اگر نمودار نخست در حالت S2 نباشد، نمودار دوم به حالت OUT بازمی‌گردد. دلیل آن نیز در دستورکار آمده است: وقتی خنک‌کننده خاموش است، تنظیم دور آن معنایی ندارد.

برچسب‌های $T > 35$ و $T < 25$ که در نمودار دوم روی یال‌های ورود و خروج OUT دیده می‌شوند، دقیقاً همان آستانه‌هایی هستند که ماشین نخست را به S2 می‌برند و از آن خارج می‌کنند. در پیاده‌سازی،

به‌جای تکرار این دو عدد، مستقیماً به وضعیّت ماشین نخست نگاه شده است؛ این صورت کلّی‌تر است و حالت‌های گرمایش و خطا را نیز به‌درستی پوشش می‌دهد.

۳.۶ تصمیم‌های طراحی

الف) نامساوی‌های اکید و پسماند

همهٔ شرط‌های دستورکار به‌صورت نامساوی اکید نوشته شده‌اند و در پیاده‌سازی نیز به همین صورت رعایت شده‌اند. بنابراین دمای دقیقاً برابر با یک آستانه، گذار ایجاد نمی‌کند: در $T = 35$ خنک‌کننده روشن نمی‌شود و تنها از $T = 36$ به بعد روشن می‌شود.

نکتهٔ مهم‌تر آنکه آستانه‌های رفت و برگشت با هم برابر نیستند و همین، پسماند[۹] می‌سازد. خنک‌کننده در $T > 35$ روشن می‌شود امّا تا $T < 25$ خاموش نمی‌شود، و گرم‌کننده در $T < 15$ روشن می‌شود امّا تا $T > 30$ خاموش نمی‌شود. این فاصله عمدی است و از کلیدزنی پیاپی گرم‌کننده و خنک‌کننده در نزدیکی یک آستانهٔ واحد جلوگیری می‌کند. یک پیامد مستقیم آن، که در آزمون مرزی هم سنجیده شده، این است که اگر سامانه در حال گرم کردن باشد، در دمای معتدل 20 درجه نیز گرم‌کننده همچنان روشن می‌ماند و تنها پس از عبور از 30 درجه خاموش می‌شود.

ب) پایش سلامت حسگر و حالت امن

دستورکار می‌گوید حسگر دمایی میان -10 تا $+60$ درجه گزارش می‌کند، امّا رفتار سامانه را در برابر مقدار خارج از این بازه مشخّص نکرده است. چون ورودی هشت‌بیتی علامت‌دار است و می‌تواند مقادیری بیرون از این بازه بگیرد، در طرح یک سیگنال اعتبار حسگر ساخته شد و بر پایهٔ آن، خروجی هشدار Error_Sensor و یک حالت امن[۱۰] به هر دو ماشین افزوده شد.

جزئیّات این تصمیم اهمّیّت دارد: اعتبار حسگر مستقیماً روی منطق خروجی اثر می‌گذارد، نه فقط روی گذار حالت. بنابراین به‌محض ظاهر شدن یک مقدار نامعتبر، گرم‌کننده و خنک‌کننده بی‌درنگ و بدون انتظار برای لبهٔ ساعت خاموش می‌شوند و پرچم خطا یک می‌شود؛ ماشین‌های حالت نیز در نخستین لبهٔ فعّال بعدی وارد حالت امن خود می‌گردند. این ترتیب برای یک واحد کنترل ایمنی‌محور رفتار درستی است، زیرا یک قرائت نامعتبر نباید حتّی برای یک پالس ساعت به فرمان گرمایش یا سرمایش بینجامد.

[9] Hysteresis

[10] Fail-Safe State

ج) هم‌گامی دو ماشین

برای تعیین فعّال بودن ماشین دور پنکه، به‌جای حالت جاری ماشین اصلی از حالت بعدی آن استفاده شده است. اگر حالت جاری ملاک قرار می‌گرفت، در همان لبه‌ای که ماشین اصلی وارد حالت سرمایش می‌شود ماشین پنکه هنوز در OUT می‌ماند و برای یک پالس ساعت، خنک‌کننده روشن ولی دور آن صفر گزارش می‌شد. با این انتخاب، خنک‌کننده و دور چهار دور بر ثانیه در یک لبه با هم فعّال می‌شوند و چنین پنجرهٔ ناسازگاری پدید نمی‌آید.

د) بازنشانی و پیمانه‌ی سطح برد

بازنشانی ناهم‌زمان[11] و فعّال‌صفر است و ماشین اصلی را به حالت S1 و ماشین پنکه را به OUT می‌برد؛ یعنی سامانه پس از بازنشانی در وضعیّت بی‌اثر و امن قرار می‌گیرد. افزون بر این، مطابق خواستهٔ دستورکار برای نمایش واکنش سامانه با چراغ‌ها، یک پیمانهٔ سطح برد نوشته شد که دما را از کلیدها می‌گیرد و خروجی‌ها را به چراغ‌ها می‌سپارد. توجّه شود که پیمانهٔ بالای پروژه‌های سنتز، خودِ واحد کنترل است و این پیمانهٔ سطح برد تنها به‌عنوان لایهٔ اتّصال به برد ارائه شده و در گزارش‌های سنتز این آزمایش وارد نشده است.

۴.۶ توصیف واحد کنترل

پیمانهٔ incubator از چهار بلوک تشکیل شده است: دو بلوک ترکیبی برای محاسبهٔ حالت بعدی هر ماشین، یک بلوک هم‌زمان برای ثبات‌های حالت، و یک بلوک ترکیبی برای منطق خروجی. جدا نگه‌داشتن منطق حالت بعدی از ثبات حالت، همان الگوی متعارف توصیف ماشین حالت است و خواندن و آزمودن طرح را ساده می‌کند.

در بلوک منطق خروجی، همهٔ خروجی‌ها نخست مقدار پیش‌فرض امن می‌گیرند و سپس بر پایهٔ حالت جاری بازنویسی می‌شوند. این سبک نوشتن، از استنتاج ناخواستهٔ قفل شفّاف جلوگیری می‌کند و تضمین می‌نماید که هر خروجی در هر شاخهٔ ممکن مقدار معیّنی دارد. خروجی دور پنکه نیز تنها هنگامی مقدار غیرصفر می‌گیرد که ماشین اصلی در حالت سرمایش باشد.

```
1  `timescale 1ns/1ps
2
3
4  module incubator (
5      input  wire                  clk,
6      input  wire                  reset_n,
7      input  wire signed [7:0]     temperature,
8
```

[11] Asynchronous Reset

```
    output reg                      heater_on,
    output reg                      cooler_on,
    output reg [3:0]                cooler_rps,
    output reg                      Error_Sensor
);


    localparam signed [7:0] TEMP_MIN = -8'sd10;
    localparam signed [7:0] TEMP_MAX =  8'sd60;

    wire sensor_valid;
    assign sensor_valid = (temperature >= TEMP_MIN) &&
                          (temperature <= TEMP_MAX);


    localparam [1:0] M_IDLE = 2'b00;
    localparam [1:0] M_COOL = 2'b01;
    localparam [1:0] M_HEAT = 2'b10;
    localparam [1:0] M_SAFE = 2'b11;

    reg [1:0] main_state;
    reg [1:0] main_next_state;


    localparam [2:0] F_OUT  = 3'd0;
    localparam [2:0] F_4    = 3'd1;
    localparam [2:0] F_6    = 3'd2;
    localparam [2:0] F_8    = 3'd3;
    localparam [2:0] F_SAFE = 3'd4;

    reg [2:0] fan_state;
    reg [2:0] fan_next_state;


    always @(*) begin
        main_next_state = main_state;

        // Invalid sensor data has the highest priority.
        if (!sensor_valid) begin
            main_next_state = M_SAFE;
        end
        else begin
            case (main_state)
                M_IDLE: begin
                    if (temperature > 8'sd35)
                        main_next_state = M_COOL;
                    else if (temperature < 8'sd15)
                        main_next_state = M_HEAT;
                    else
                        main_next_state = M_IDLE;
                end

                M_COOL: begin
                    if (temperature < 8'sd25)
                        main_next_state = M_IDLE;
                    else
                        main_next_state = M_COOL;
                end

                M_HEAT: begin
                    if (temperature > 8'sd30)
                        main_next_state = M_IDLE;
                    else
                        main_next_state = M_HEAT;
                end

                M_SAFE: begin

                    if (temperature > 8'sd35)
                        main_next_state = M_COOL;
                    else if (temperature < 8'sd15)
                        main_next_state = M_HEAT;
```

```
                    else
                        main_next_state = M_IDLE;
                end

                default: begin
                    // Illegal internal state -> fail safe.
                    main_next_state = M_SAFE;
                end
            endcase
        end
    end


    always @(*) begin
        fan_next_state = fan_state;

        if (!sensor_valid) begin
            fan_next_state = F_SAFE;
        end
        else if (main_next_state != M_COOL) begin

            fan_next_state = F_OUT;
        end
        else begin
            case (fan_state)
                F_OUT: begin
                    fan_next_state = F_4;
                end

                F_4: begin
                    if (temperature > 8'sd40)
                        fan_next_state = F_6;
                    else
                        fan_next_state = F_4;
                end

                F_6: begin
                    if (temperature > 8'sd45)
                        fan_next_state = F_8;
                    else if (temperature < 8'sd35)
                        fan_next_state = F_4;
                    else
                        fan_next_state = F_6;
                end

                F_8: begin
                    if (temperature < 8'sd40)
                        fan_next_state = F_6;
                    else
                        fan_next_state = F_8;
                end

                F_SAFE: begin

                    fan_next_state = F_4;
                end

                default: begin
                    fan_next_state = F_SAFE;
                end
            endcase
        end
    end

    // ------------------------------------------------------------------------
    // State registers - asynchronous active-low reset
    // ------------------------------------------------------------------------
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            main_state <= M_IDLE;
            fan_state  <= F_OUT;
        end
```

```
        else begin
            main_state <= main_next_state;
            fan_state  <= fan_next_state;
        end
    end

    // ------------------------------------------------------------------------
    // Output logic
    //
    // IMPORTANT SAFE-STATE DETAIL:
    // sensor_valid directly overrides the actuator outputs. Therefore, as soon
    // as an invalid sensor value appears, Heater and Cooler are forced OFF and
    // Error_Sensor becomes 1 without waiting for another clock edge. The FSMs
    // also enter their SAFE states at the next active clock edge.
    // ------------------------------------------------------------------------
    always @(*) begin
        heater_on   = 1'b0;
        cooler_on   = 1'b0;
        cooler_rps  = 4'd0;
        Error_Sensor = ~sensor_valid;

        if (reset_n && sensor_valid) begin
            case (main_state)
                M_COOL: begin
                    cooler_on = 1'b1;
                end

                M_HEAT: begin
                    heater_on = 1'b1;
                end

                default: begin
                    heater_on = 1'b0;
                    cooler_on = 1'b0;
                end
            endcase

            // Fan output is meaningful only when cooling is active.
            if (main_state == M_COOL) begin
                case (fan_state)
                    F_4: cooler_rps = 4'd4;
                    F_6: cooler_rps = 4'd6;
                    F_8: cooler_rps = 4'd8;
                    default: cooler_rps = 4'd0;
                endcase
            end
        end
    end

endmodule
```

کد ۱: واحد کنترل انکوباتور با دو ماشین حالت

پیمانهٔ سطح برد بسیار کوتاه است و تنها ورودی هشت‌بیتی کلیدها را به‌صورت علامت‌دار تفسیر و خروجی‌های واحد کنترل را به چراغ‌ها متّصل می‌کند.

```
`timescale 1ns/1ps

module incubator_board_top (
    input  wire             CLOCK_IN,
    input  wire             RESET_N,
    input  wire [7:0]       SW_TEMP,

    output wire             LED_HEATER,
    output wire             LED_COOLER,
    output wire [3:0]       LED_RPS,
```

```
    output wire              LED_ERROR
);

    wire signed [7:0] signed_temperature;
    assign signed_temperature = SW_TEMP;

    incubator core (
        .clk          (CLOCK_IN),
        .reset_n      (RESET_N),
        .temperature  (signed_temperature),
        .heater_on    (LED_HEATER),
        .cooler_on    (LED_COOLER),
        .cooler_rps   (LED_RPS),
        .Error_Sensor (LED_ERROR)
    );

endmodule
```

کد ۲: پیمانه‌ی سطح برد برای نمایش خروجی‌ها با چراغ‌ها

۵.۶ میزهای آزمون

الف) شبیه‌سازی حلقه‌بسته با مدل فرایند

آزمودن یک واحد کنترل با اعمال دمای دلخواه، تنها بخشی از رفتار آن را می‌سنجد. برای آنکه سامانه در شرایطی نزدیک به واقعیّت ارزیابی شود، در میز آزمون نخست یک مدل سادهٔ فرایند ساخته شد که دمای محفظه را بر پایهٔ فرمان‌های خود واحد کنترل به‌روز می‌کند. بدین‌ترتیب حلقهٔ کنترل بسته می‌شود: واحد کنترل بر دما اثر می‌گذارد و دما دوباره ورودی واحد کنترل می‌شود.

هر «دقیقه» در این مدل معادل ۱۰۰ نانوثانیه گرفته شده است و در پایان هر دقیقه، تغییر دما از جمع سه سهم به دست می‌آید: یک بار گرمایی بیرونی که میز آزمون آن را تعیین می‌کند، سهم گرم‌کننده در صورت روشن بودن، و سهم خنک‌کننده که به دور پنکه بستگی دارد. مقادیر عددی این سهم‌ها انتخاب خود میز آزمون‌اند و بخشی از خواستهٔ دستورکار نیستند؛ تنها نقش آن‌ها ساختن یک فرایند گرمایی باورپذیر است. دمای مدل نیز در همان بازهٔ -10 تا $+60$ محدود می‌شود تا از بازهٔ معتبر حسگر بیرون نزند.

افزون بر مدل فرایند، این میز آزمون امکان تزریق خطا[۱۲] نیز دارد: با فعّال کردن یک کلید داخلی، مقدار حسگر با یک عدد نامعتبر جایگزین می‌شود تا رفتار حالت امن آزموده شود. یک زمان‌سنج نگهبان[۱۳] نیز در نظر گرفته شده تا اگر گذاری که آزمون منتظر آن است هرگز رخ ندهد، شبیه‌سازی به‌جای معلّق ماندن با پیام روشن پایان یابد.

```
`timescale 1ns/1ps


module incubator_tb;

```

[12] Fault Injection

[13] Watchdog Timeout

```
    reg clk;
    reg reset_n;


    integer plant_temperature;
    integer external_heat_load;
    integer delta_temperature;
    integer simulated_minute;
    integer errors;


    reg fault_enable;
    reg signed [7:0] fault_temperature;

    wire signed [7:0] temperature;
    wire heater_on;
    wire cooler_on;
    wire [3:0] cooler_rps;
    wire Error_Sensor;

    assign temperature = fault_enable ? fault_temperature : plant_temperature;

    incubator dut (
        .clk          (clk),
        .reset_n      (reset_n),
        .temperature  (temperature),
        .heater_on    (heater_on),
        .cooler_on    (cooler_on),
        .cooler_rps   (cooler_rps),
        .Error_Sensor (Error_Sensor)
    );


    always #5 clk = ~clk;


    always begin
        #100;

        if (reset_n) begin
            delta_temperature = external_heat_load;

            if (heater_on)
                delta_temperature = delta_temperature + 3;

            if (cooler_on) begin
                case (cooler_rps)
                    4'd4: delta_temperature = delta_temperature - 1;
                    4'd6: delta_temperature = delta_temperature - 2;
                    4'd8: delta_temperature = delta_temperature - 6;
                    default: delta_temperature = delta_temperature;
                endcase
            end

            plant_temperature = plant_temperature + delta_temperature;


            if (plant_temperature > 60)
                plant_temperature = 60;
            else if (plant_temperature < -10)
                plant_temperature = -10;

            simulated_minute = simulated_minute + 1;

            $display("minute=%0d  plantT=%0d  sensedT=%0d  H=%b C=%b RPS=%0d ERR=%b extHeat=%0d",
                     simulated_minute, plant_temperature, $signed(temperature),
                     heater_on, cooler_on, cooler_rps, Error_Sensor,
                     external_heat_load);
        end
    end


```

```
    task check_outputs_safe;
    begin
        if ((Error_Sensor !== 1'b1) ||
            (heater_on !== 1'b0) ||
            (cooler_on !== 1'b0) ||
            (cooler_rps !== 4'd0)) begin
            $display("[ERROR] Safe-state outputs are incorrect at time %0t", $time);
            errors = errors + 1;
        end
        else begin
            $display("[PASS] Safe-state outputs are correct at time %0t", $time);
        end
    end
    endtask


    initial begin
        clk               = 1'b0;
        reset_n           = 1'b0;
        plant_temperature = 10;   // cold start -> heater must turn ON
        external_heat_load = 0;
        simulated_minute  = 0;
        fault_enable      = 1'b0;
        fault_temperature = 8'sd0;
        errors            = 0;

        $display("============================================================");
        $display("Experiment 6 smart closed-loop simulation started");
        $display("============================================================");


        #27;
        reset_n = 1'b1;


        wait (heater_on === 1'b1);
        $display("[PASS] Heater activated automatically. T=%0d", $signed(temperature));


        wait (heater_on === 1'b0);
        $display("[PASS] Heater turned OFF after crossing 30 C. T=%0d", $signed(temperature));

        external_heat_load = 4;
        $display("[INFO] External heat load applied: +4 C/min");


        wait (cooler_on === 1'b1);
        wait (cooler_rps == 4'd4);
        $display("[PASS] Cooler activated and fan entered 4 RPS.");


        wait (cooler_rps == 4'd6);
        $display("[PASS] Fan increased to 6 RPS.");

        wait (cooler_rps == 4'd8);
        $display("[PASS] Fan increased to 8 RPS.");


        external_heat_load = 0;
        $display("[INFO] External heat load removed.");

        wait (cooler_rps == 4'd6);
        $display("[PASS] Fan decreased from 8 to 6 RPS.");

        wait (cooler_rps == 4'd4);
        $display("[PASS] Fan decreased from 6 to 4 RPS.");

        wait (cooler_on === 1'b0);
        if (cooler_rps !== 4'd0) begin
            $display("[ERROR] Fan must be OUT when cooler turns OFF.");
            errors = errors + 1;
        end
```

```
        else begin
            $display("[PASS] Cooler turned OFF below 25 C and fan returned OUT.");
        end

        #20;
        fault_temperature = 8'sd70;
        fault_enable = 1'b1;
        #1;
        $display("[INFO] Injected sensor fault: +70 C");
        check_outputs_safe;


        #40;
        if (dut.main_state !== 2'b11) begin
            $display("[ERROR] Main FSM did not enter M_SAFE for +70 C.");
            errors = errors + 1;
        end
        else begin
            $display("[PASS] Main FSM entered M_SAFE for +70 C.");
        end

        fault_enable = 1'b0;
        #30;
        if (Error_Sensor !== 1'b0) begin
            $display("[ERROR] Error_Sensor did not clear after valid recovery.");
            errors = errors + 1;
        end
        else begin
            $display("[PASS] Sensor recovered from high fault.");
        end


        fault_temperature = -8'sd20;
        fault_enable = 1'b1;
        #1;
        $display("[INFO] Injected sensor fault: -20 C");
        check_outputs_safe;

        #40;
        if (dut.main_state !== 2'b11) begin
            $display("[ERROR] Main FSM did not enter M_SAFE for -20 C.");
            errors = errors + 1;
        end
        else begin
            $display("[PASS] Main FSM entered M_SAFE for -20 C.");
        end

        fault_enable = 1'b0;
        #30;

        if (errors == 0) begin
            $display("============================================================");
            $display("ALL AUTOMATIC CLOSED-LOOP TESTS PASSED");
            $display("============================================================");
        end
        else begin
            $display("============================================================");
            $display("SIMULATION FINISHED WITH %0d ERROR(S)", errors);
            $display("============================================================");
        end

        #20;
        $finish;
    end


    initial begin
        #10000;
        $display("[FATAL] Testbench timeout - a required state transition was not reached.");
        $finish;
    end

```

```
endmodule
```

کد ۳: میز آزمون حلقه‌بسته با مدل فرایند و تزریق خطای حسگر

ب) آزمون هدفمند مرزها

شبیه‌سازی حلقه‌بسته نشان می‌دهد سامانه در عمل درست کار می‌کند، امّا از هر آستانه فقط از یک سو عبور می‌کند و مقدار دقیقاً برابر با آستانه را نمی‌آزماید. ازاین‌رو میز آزمون دوم ساخته شد که دما را مستقیماً و به‌صورت پلّه‌ای اعمال می‌کند و پس از هر پلّه، هر چهار خروجی را با مقدار انتظاری می‌سنجد.
این آزمون برای هر آستانه دو مقدار می‌آزماید: مقدار دقیقاً برابر با آستانه که نباید گذاری ایجاد کند، و نخستین مقدار پس از آن که باید گذار را ایجاد کند. هر پلّه سه لبهٔ ساعت نگه داشته می‌شود، زیرا یک تغییر دمای بزرگ می‌تواند به بیش از یک گذار پیاپی در ماشین پنکه بینجامد و خروجی باید پس از آرام گرفتن هر دو ماشین سنجیده شود.

```
`timescale 1ns/1ps


module incubator_tb_edges;

    reg clk;
    reg reset_n;
    reg signed [7:0] temperature;
    wire heater_on;
    wire cooler_on;
    wire [3:0] cooler_rps;
    wire Error_Sensor;
    integer errors;

    incubator dut (
        .clk          (clk),
        .reset_n      (reset_n),
        .temperature  (temperature),
        .heater_on    (heater_on),
        .cooler_on    (cooler_on),
        .cooler_rps   (cooler_rps),
        .Error_Sensor (Error_Sensor)
    );

    always #5 clk = ~clk;

    task apply_temp;
        input signed [7:0] t;
    begin
        temperature = t;
        // Hold long enough for both FSMs to settle across sequential states.
        repeat (3) @(posedge clk);
        #1;
    end
    endtask

    task check_expected;
        input exp_heater;
        input exp_cooler;
        input [3:0] exp_rps;
        input exp_error;
    begin
        if ((heater_on !== exp_heater) ||
```

```
            (cooler_on !== exp_cooler) ||
            (cooler_rps !== exp_rps) ||
            (Error_Sensor !== exp_error)) begin
            $display("[ERROR] T=%0d got H=%b C=%b RPS=%0d ERR=%b | expected H=%b C=%b RPS=%0d
 ERR=%b",
                     $signed(temperature), heater_on, cooler_on, cooler_rps, Error_Sensor,
                     exp_heater, exp_cooler, exp_rps, exp_error);
            errors = errors + 1;
        end
        else begin
            $display("[PASS]  T=%0d -> H=%b C=%b RPS=%0d ERR=%b",
                     $signed(temperature), heater_on, cooler_on, cooler_rps, Error_Sensor);
        end
    end
    endtask

    initial begin
        clk = 0;
        reset_n = 0;
        temperature = 8'sd20;
        errors = 0;

        #12;
        reset_n = 1;
        repeat (2) @(posedge clk);

        // Main FSM: equality must NOT trigger strict transitions.
        apply_temp(8'sd35); check_expected(0, 0, 0, 0);  // not >35
        apply_temp(8'sd36); check_expected(0, 1, 4, 0);  // enter cooling

        // Fan thresholds.
        apply_temp(8'sd40); check_expected(0, 1, 4, 0);  // not >40
        apply_temp(8'sd41); check_expected(0, 1, 6, 0);
        apply_temp(8'sd45); check_expected(0, 1, 6, 0);  // not >45
        apply_temp(8'sd46); check_expected(0, 1, 8, 0);
        apply_temp(8'sd40); check_expected(0, 1, 8, 0);  // not <40
        apply_temp(8'sd39); check_expected(0, 1, 6, 0);
        apply_temp(8'sd35); check_expected(0, 1, 6, 0);  // not <35
        apply_temp(8'sd34); check_expected(0, 1, 4, 0);
        apply_temp(8'sd25); check_expected(0, 1, 4, 0);  // not <25
        apply_temp(8'sd24); check_expected(0, 0, 0, 0);  // cooler OFF

        // Heater thresholds.
        apply_temp(8'sd15); check_expected(0, 0, 0, 0);  // not <15
        apply_temp(8'sd14); check_expected(1, 0, 0, 0);
        apply_temp(8'sd30); check_expected(1, 0, 0, 0);  // not >30
        apply_temp(8'sd31); check_expected(0, 0, 0, 0);

        // Sensor health boundaries.
        apply_temp(8'sd60);  check_expected(0, 1, 8, 0); // +60 is valid; cooling reaches 8 RPS
        temperature = 8'sd61; #1; check_expected(0, 0, 0, 1); // invalid high: immediate safe outputs
        // The fault must be held across at least one active edge, otherwise the
        // FSMs never leave their previous state and only the combinational
        // output override is exercised.
        repeat (3) @(posedge clk); #1;
        if (dut.main_state !== 2'b11) begin
            $display("[ERROR] Main FSM did not enter M_SAFE for the high fault.");
            errors = errors + 1;
        end
        apply_temp(8'sd20);  check_expected(0, 0, 0, 0); // recovery from SAFE -> IDLE

        apply_temp(-8'sd10); check_expected(1, 0, 0, 0); // -10 is valid and requires heating
        temperature = -8'sd11; #1; check_expected(0, 0, 0, 1); // invalid low
        repeat (3) @(posedge clk); #1;
        if (dut.main_state !== 2'b11) begin
            $display("[ERROR] Main FSM did not enter M_SAFE for the low fault.");
            errors = errors + 1;
        end
        apply_temp(8'sd20);  check_expected(0, 0, 0, 0); // recovery from SAFE -> IDLE

        // Heating hysteresis: once heating has started, it must persist until
        // the temperature actually exceeds 30 C, even at a comfortable 20 C.
```

```
        apply_temp(8'sd14); check_expected(1, 0, 0, 0);
        apply_temp(8'sd20); check_expected(1, 0, 0, 0);
        apply_temp(8'sd31); check_expected(0, 0, 0, 0);

        if (errors == 0)
            $display("ALL DIRECTED EDGE TESTS PASSED");
        else
            $display("DIRECTED TEST FINISHED WITH %0d ERROR(S)", errors);

        #20;
        $finish;
    end

endmodule
```

کد ۴: آزمون هدفمند آستانه‌ها و مرزهای اعتبار حسگر

جدول زیر، بیست و پنج وارسی این آزمون و منطق پشت هر یک را خلاصه می‌کند.

آنچه سنجیده می‌شود	خطا	دور	خنک‌کننده	گرم‌کننده	T
برابری با آستانه گذار نمی‌سازد	0	0	0	0	35
ورود به سرمایش و دور اوّلیّه	0	4	1	0	36
برابری با آستانه گذار نمی‌سازد	0	4	1	0	40
افزایش دور به شش	0	6	1	0	41
برابری با آستانه گذار نمی‌سازد	0	6	1	0	45
افزایش دور به هشت	0	8	1	0	46
برابری با آستانه گذار نمی‌سازد	0	8	1	0	40
کاهش دور به شش	0	6	1	0	39
برابری با آستانه گذار نمی‌سازد	0	6	1	0	35
کاهش دور به چهار	0	4	1	0	34
برابری با آستانه گذار نمی‌سازد	0	4	1	0	25
خاموشی خنک‌کننده و بازگشت پنکه به OUT	0	0	0	0	24
برابری با آستانه گذار نمی‌سازد	0	0	0	0	15
روشن شدن گرم‌کننده	0	0	0	1	14
برابری با آستانه گذار نمی‌سازد	0	0	0	1	30
خاموشی گرم‌کننده	0	0	0	0	31
بیشترین دمای معتبر	0	8	1	0	60
خطای حسگر و خروجی امن بی‌درنگ	1	0	0	0	61
بازگشت از حالت امن	0	0	0	0	20
کمترین دمای معتبر	0	0	0	1	-10
خطای حسگر در سمت پایین	1	0	0	0	-11
بازگشت از حالت امن	0	0	0	0	20
آغاز گرمایش برای آزمون پسماند	0	0	0	1	14
پسماند: گرمایش تا عبور از ۳۰ ادامه می‌یابد	0	0	0	1	20
پایان گرمایش پس از عبور از آستانه	0	0	0	0	31

جدول ۳: وارسی‌های آزمون هدفمند مرزها و هدف هر یک

دو وارسی پایانی مربوط به تزریق خطا نیز شایان توضیح‌اند. خطا باید دست‌کم یک لبهٔ فعّال ساعت نگه داشته شود تا ماشین‌ها واقعاً وارد حالت امن شوند؛ اگر خطا کوتاه‌تر از یک لبه باشد، تنها منطق ترکیبی خروجی اثر می‌پذیرد و ثبات حالت دست‌نخورده می‌ماند. در آزمون، پس از تزریق خطا نخست خروجی‌های امن بی‌درنگ سنجیده می‌شوند و سپس، پس از گذشت چند لبه، ورود ماشین اصلی به حالت امن نیز مستقیماً وارسی می‌گردد.

۶.۶ اجرا در ModelSim

برای هر یک از دو میز آزمون یک پروندهٔ فرمان نوشته شد تا اجرا تکرارپذیر باشد. پروندهٔ نخست کتابخانه را از نو می‌سازد، طرح و میز آزمون حلقه‌بسته را همگردانی می‌کند، شبیه‌ساز را بارگذاری و پنجرهٔ موج را از روی پروندهٔ جداگانهٔ آن می‌سازد و شبیه‌سازی را تا پایان اجرا می‌کند.

```
transcript on
if {[file exists work]} {
    vdel -lib work -all
}
vlib work
vmap work work
vlog ../src/incubator.v
vlog incubator_tb.v
vsim -voptargs=+acc work.incubator_tb
do wave.do
run -all
wave zoom full
```

کد ۵: پرونده‌ی فرمان اجرای شبیه‌سازی حلقه‌بسته

```
quietly WaveActivateNextPane {} 0
add wave -divider {INPUTS}
add wave -radix binary sim:/incubator_tb/clk
add wave -radix binary sim:/incubator_tb/reset_n
add wave -radix decimal sim:/incubator_tb/temperature
add wave -radix decimal sim:/incubator_tb/plant_temperature
add wave -radix decimal sim:/incubator_tb/external_heat_load
add wave -radix binary sim:/incubator_tb/fault_enable
add wave -divider {OUTPUTS}
add wave -radix binary sim:/incubator_tb/heater_on
add wave -radix binary sim:/incubator_tb/cooler_on
add wave -radix unsigned sim:/incubator_tb/cooler_rps
add wave -radix binary sim:/incubator_tb/Error_Sensor
add wave -divider {INTERNAL FSM STATES}
add wave -radix unsigned sim:/incubator_tb/dut/main_state
add wave -radix unsigned sim:/incubator_tb/dut/main_next_state
add wave -radix unsigned sim:/incubator_tb/dut/fan_state
add wave -radix unsigned sim:/incubator_tb/dut/fan_next_state
configure wave -namecolwidth 260
configure wave -valuecolwidth 100
configure wave -timelineunits ns
WaveRestoreZoom {0 ns} {3500 ns}
```

کد ۶: پرونده‌ی پیکربندی پنجره‌ی موج

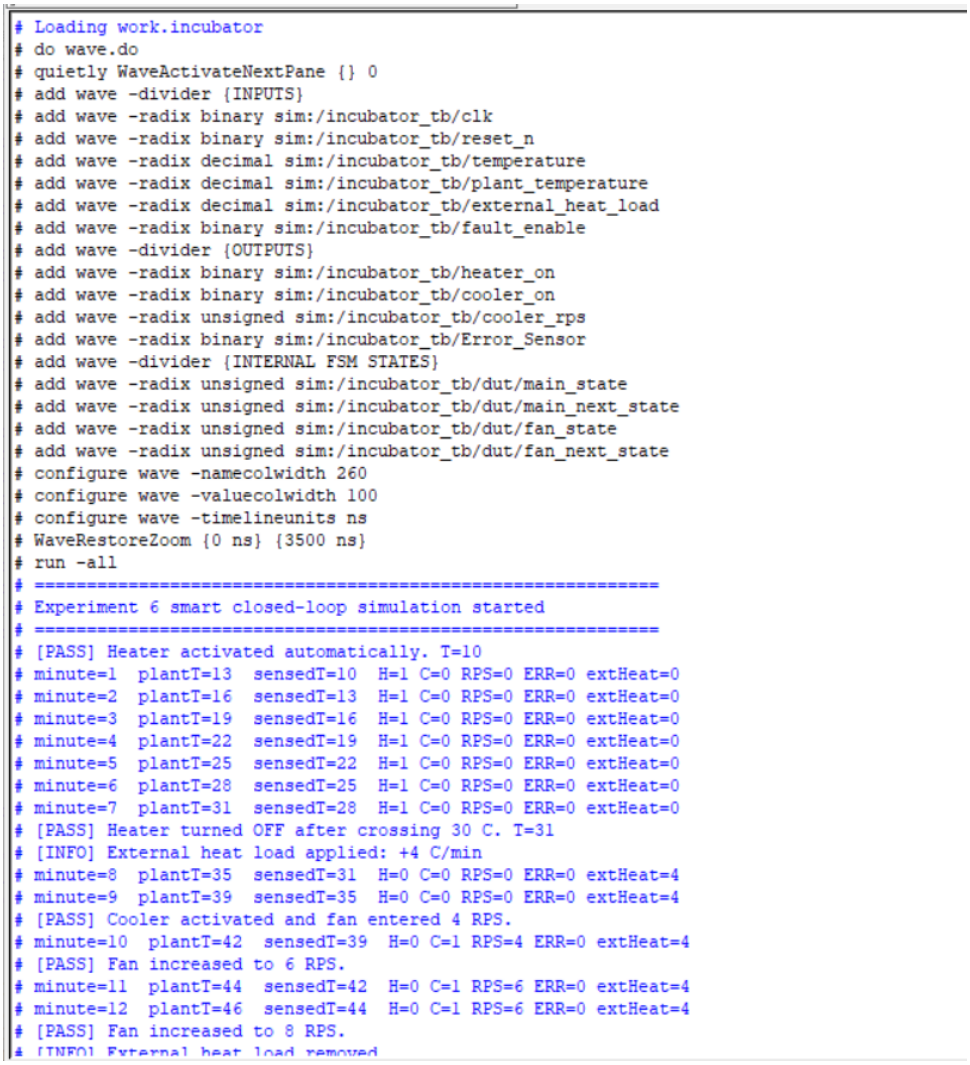
پروندهٔ دوم همان طرح را با میز آزمون مرزی اجرا می‌کند. چون در این آزمون تنها به خروجی متنی و چند سیگنال کلیدی نیاز است، سیگنال‌ها مستقیماً در همین پرونده افزوده شده‌اند.

```
transcript on
if {[file exists work]} {
    vdel -lib work -all
}
vlib work
vmap work work
vlog ../src/incubator.v
vlog incubator_tb_edges.v
```

```
vsim -voptargs=+acc work.incubator_tb_edges
add wave -radix decimal sim:/incubator_tb_edges/temperature
add wave sim:/incubator_tb_edges/heater_on
add wave sim:/incubator_tb_edges/cooler_on
add wave -radix unsigned sim:/incubator_tb_edges/cooler_rps
add wave sim:/incubator_tb_edges/Error_Sensor
add wave -radix unsigned sim:/incubator_tb_edges/dut/main_state
add wave -radix unsigned sim:/incubator_tb_edges/dut/fan_state
run -all
wave zoom full
```

کد ۷: پرونده‌ی فرمان اجرای آزمون هدفمند مرزها

تصویر زیر آغاز اجرا را نشان می‌دهد: بارگذاری شبیه‌ساز، افزودن سیگنال‌ها به پنجرهٔ موج در سه دستهٔ ورودی، خروجی و حالت‌های درونی، و سپس نخستین سطرهای خروجی آزمون.



```
# Loading work.incubator
# do wave.do
# quietly WaveActivateNextPane {} 0
# add wave -divider {INPUTS}
# add wave -radix binary sim:/incubator_tb/clk
# add wave -radix binary sim:/incubator_tb/reset_n
# add wave -radix decimal sim:/incubator_tb/temperature
# add wave -radix decimal sim:/incubator_tb/plant_temperature
# add wave -radix decimal sim:/incubator_tb/external_heat_load
# add wave -radix binary sim:/incubator_tb/fault_enable
# add wave -divider {OUTPUTS}
# add wave -radix binary sim:/incubator_tb/heater_on
# add wave -radix binary sim:/incubator_tb/cooler_on
# add wave -radix unsigned sim:/incubator_tb/cooler_rps
# add wave -radix binary sim:/incubator_tb/Error_Sensor
# add wave -divider {INTERNAL FSM STATES}
# add wave -radix unsigned sim:/incubator_tb/dut/main_state
# add wave -radix unsigned sim:/incubator_tb/dut/main_next_state
# add wave -radix unsigned sim:/incubator_tb/dut/fan_state
# add wave -radix unsigned sim:/incubator_tb/dut/fan_next_state
# configure wave -namecolwidth 260
# configure wave -valuecolwidth 100
# configure wave -timelineunits ns
# WaveRestoreZoom {0 ns} {3500 ns}
# run -all
# ============================================================
# Experiment 6 smart closed-loop simulation started
# ============================================================
# [PASS] Heater activated automatically. T=10
# minute=1  plantT=13  sensedT=10  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=2  plantT=16  sensedT=13  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=3  plantT=19  sensedT=16  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=4  plantT=22  sensedT=19  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=5  plantT=25  sensedT=22  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=6  plantT=28  sensedT=25  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=7  plantT=31  sensedT=28  H=1 C=0 RPS=0 ERR=0 extHeat=0
# [PASS] Heater turned OFF after crossing 30 C. T=31
# [INFO] External heat load applied: +4 C/min
# minute=8  plantT=35  sensedT=31  H=0 C=0 RPS=0 ERR=0 extHeat=4
# minute=9  plantT=39  sensedT=35  H=0 C=0 RPS=0 ERR=0 extHeat=4
# [PASS] Cooler activated and fan entered 4 RPS.
# minute=10  plantT=42  sensedT=39  H=0 C=1 RPS=4 ERR=0 extHeat=4
# [PASS] Fan increased to 6 RPS.
# minute=11  plantT=44  sensedT=42  H=0 C=1 RPS=6 ERR=0 extHeat=4
# minute=12  plantT=46  sensedT=44  H=0 C=1 RPS=6 ERR=0 extHeat=4
# [PASS] Fan increased to 8 RPS.
# [INFO] External heat load removed
```

شکل ۱: بارگذاری شبیه‌ساز، پیکربندی پنجره‌ی موج و آغاز اجرای شبیه‌سازی

۷.۶ تحلیل رونوشت و شکل موج

الف) سیگنال‌های پنجره‌ی موج

جدول زیر سیزده سیگنال نمایش‌داده‌شده در پنجرهٔ موج و معنای آن‌ها را گردآوری می‌کند.

دسته	سیگنال	معنا
ورودی	clk	ساعت با دورهٔ ۱۰ نانوثانیه
ورودی	reset_n	بازنشانی ناهم‌زمان فعّال‌صفر
ورودی	temperature	دمایی که واحد کنترل می‌بیند
ورودی	plant_temperature	دمای واقعی مدل محفظه
ورودی	external_heat_load	بار گرمایی بیرونی بر حسب درجه در دقیقه
ورودی	fault_enable	فعّال‌ساز تزریق خطای حسگر
خروجی	heater_on	فرمان گرم‌کننده
خروجی	cooler_on	فرمان خنک‌کننده
خروجی	cooler_rps	دور پنکه بر حسب دور بر ثانیه
خروجی	Error_Sensor	هشدار نامعتبر بودن دادهٔ حسگر
درونی	main_state	حالت جاری ماشین اصلی
درونی	main_next_state	حالت بعدی ماشین اصلی
درونی	fan_state	حالت جاری ماشین دور پنکه

جدول ۴: سیگنال‌های پنجره‌ی موج و معنای آن‌ها

ب) دنباله‌ی کامل شبیه‌سازی حلقه‌بسته

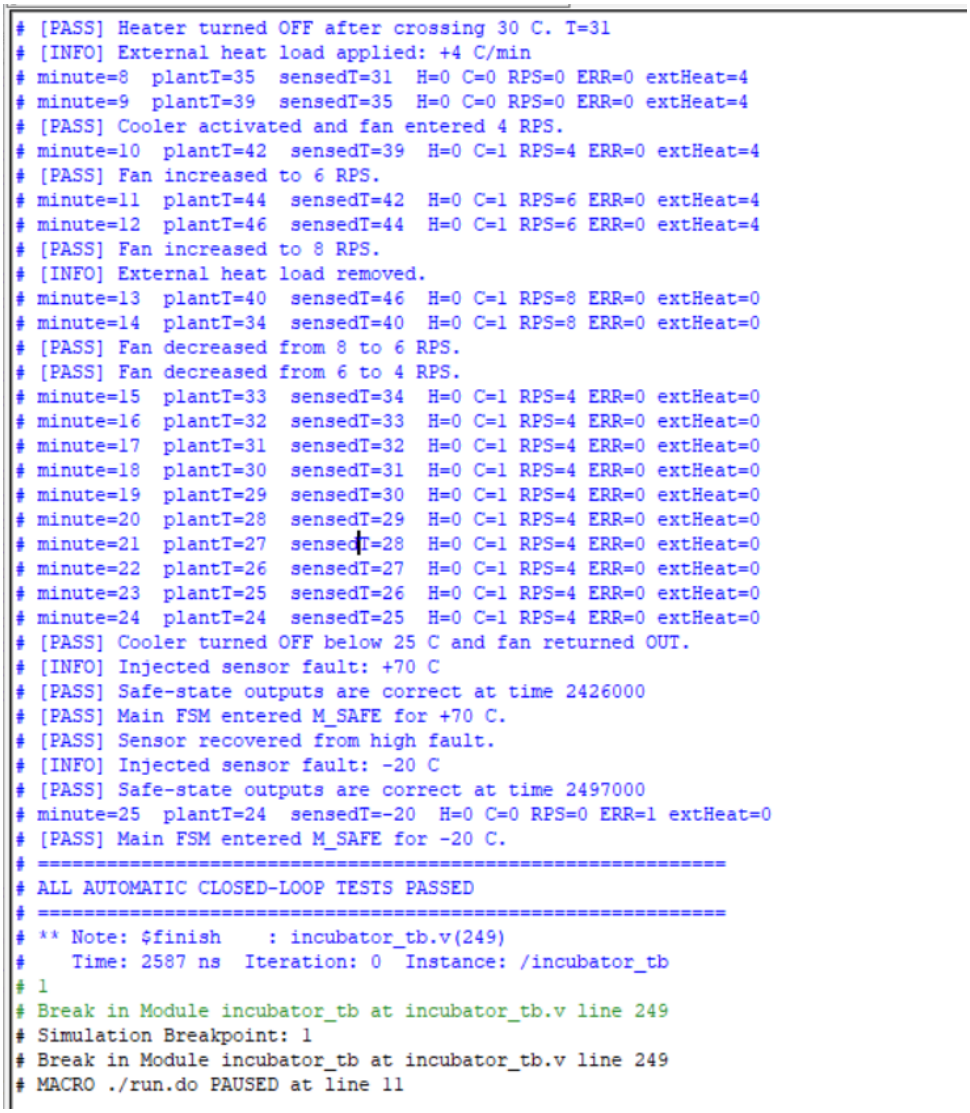
رونوشت اجرا برای هر دقیقهٔ شبیه‌سازی یک سطر چاپ می‌کند. در خواندن این سطرها یک نکته اهمّیّت دارد: دمای حس‌شده همان مقداری است که واحد کنترل در *طول آن دقیقه* بر پایهٔ آن تصمیم گرفته است، حال آنکه دمای محفظه، مقدار *پایان* همان دقیقه پس از اثر گرم‌کننده و خنک‌کننده است. بنابراین یک دقیقه اختلاف میان این دو ستون، نه خطا بلکه همان تأخیر طبیعی فرایند است.

دقیقه	دمای حس‌شده	گرم‌کننده	خنک‌کننده	دور	بار بیرونی	دمای پایان دقیقه
1	10	1	0	0	0	13
2	13	1	0	0	0	16
3	16	1	0	0	0	19
4	19	1	0	0	0	22
5	22	1	0	0	0	25
6	25	1	0	0	0	28
7	28	1	0	0	0	31
8	31	0	0	0	4	35
9	35	0	0	0	4	39
10	39	0	1	4	4	42
11	42	0	1	6	4	44
12	44	0	1	6	4	46
13	46	0	1	8	0	40
14	40	0	1	8	0	34
15	34	0	1	4	0	33
16	33	0	1	4	0	32
17	32	0	1	4	0	31
18	31	0	1	4	0	30
19	30	0	1	4	0	29
20	29	0	1	4	0	28
21	28	0	1	4	0	27
22	27	0	1	4	0	26
23	26	0	1	4	0	25
24	25	0	1	4	0	24
25	−20	0	0	0	0	24

جدول ۵: دنباله‌ی کامل شبیه‌سازی حلقه‌بسته؛ دقیقه‌ی **۲۵** در حالت تزریق خطاست

```
# run -all
# ============================================================
# Experiment 6 smart closed-loop simulation started
# ============================================================
# [PASS] Heater activated automatically. T=10
# minute=1  plantT=13  sensedT=10  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=2  plantT=16  sensedT=13  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=3  plantT=19  sensedT=16  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=4  plantT=22  sensedT=19  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=5  plantT=25  sensedT=22  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=6  plantT=28  sensedT=25  H=1 C=0 RPS=0 ERR=0 extHeat=0
# minute=7  plantT=31  sensedT=28  H=1 C=0 RPS=0 ERR=0 extHeat=0
# [PASS] Heater turned OFF after crossing 30 C. T=31
# [INFO] External heat load applied: +4 C/min
# minute=8  plantT=35  sensedT=31  H=0 C=0 RPS=0 ERR=0 extHeat=4
# minute=9  plantT=39  sensedT=35  H=0 C=0 RPS=0 ERR=0 extHeat=4
# [PASS] Cooler activated and fan entered 4 RPS.
# minute=10  plantT=42  sensedT=39  H=0 C=1 RPS=4 ERR=0 extHeat=4
# [PASS] Fan increased to 6 RPS.
# minute=11  plantT=44  sensedT=42  H=0 C=1 RPS=6 ERR=0 extHeat=4
# minute=12  plantT=46  sensedT=44  H=0 C=1 RPS=6 ERR=0 extHeat=4
# [PASS] Fan increased to 8 RPS.
# [INFO] External heat load removed.
# minute=13  plantT=40  sensedT=46  H=0 C=1 RPS=8 ERR=0 extHeat=0
# minute=14  plantT=34  sensedT=40  H=0 C=1 RPS=8 ERR=0 extHeat=0
# [PASS] Fan decreased from 8 to 6 RPS.
# [PASS] Fan decreased from 6 to 4 RPS.
# minute=15  plantT=33  sensedT=34  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=16  plantT=32  sensedT=33  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=17  plantT=31  sensedT=32  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=18  plantT=30  sensedT=31  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=19  plantT=29  sensedT=30  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=20  plantT=28  sensedT=29  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=21  plantT=27  sensedT=28  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=22  plantT=26  sensedT=27  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=23  plantT=25  sensedT=26  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=24  plantT=24  sensedT=25  H=0 C=1 RPS=4 ERR=0 extHeat=0
# [PASS] Cooler turned OFF below 25 C and fan returned OUT.
# [INFO] Injected sensor fault: +70 C
# [PASS] Safe-state outputs are correct at time 2426000
# [PASS] Main FSM entered M_SAFE for +70 C.
# [PASS] Sensor recovered from high fault.
# [INFO] Injected sensor fault: -20 C
# [PASS] Safe-state outputs are correct at time 2497000
# minute=25  plantT=24  sensedT=-20  H=0 C=0 RPS=0 ERR=1 extHeat=0
# [PASS] Main FSM entered M_SAFE for -20 C.
# ============================================================
```

شکل ۲: رونوشت شبیه‌سازی حلقه‌بسته از راه‌اندازی سرد تا پایان تزریق خطاها



```
# [PASS] Heater turned OFF after crossing 30 C. T=31
# [INFO] External heat load applied: +4 C/min
# minute=8  plantT=35  sensedT=31  H=0 C=0 RPS=0 ERR=0 extHeat=4
# minute=9  plantT=39  sensedT=35  H=0 C=0 RPS=0 ERR=0 extHeat=4
# [PASS] Cooler activated and fan entered 4 RPS.
# minute=10  plantT=42  sensedT=39  H=0 C=1 RPS=4 ERR=0 extHeat=4
# [PASS] Fan increased to 6 RPS.
# minute=11  plantT=44  sensedT=42  H=0 C=1 RPS=6 ERR=0 extHeat=4
# minute=12  plantT=46  sensedT=44  H=0 C=1 RPS=6 ERR=0 extHeat=4
# [PASS] Fan increased to 8 RPS.
# [INFO] External heat load removed.
# minute=13  plantT=40  sensedT=46  H=0 C=1 RPS=8 ERR=0 extHeat=0
# minute=14  plantT=34  sensedT=40  H=0 C=1 RPS=8 ERR=0 extHeat=0
# [PASS] Fan decreased from 8 to 6 RPS.
# [PASS] Fan decreased from 6 to 4 RPS.
# minute=15  plantT=33  sensedT=34  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=16  plantT=32  sensedT=33  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=17  plantT=31  sensedT=32  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=18  plantT=30  sensedT=31  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=19  plantT=29  sensedT=30  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=20  plantT=28  sensedT=29  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=21  plantT=27  sensedT=28  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=22  plantT=26  sensedT=27  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=23  plantT=25  sensedT=26  H=0 C=1 RPS=4 ERR=0 extHeat=0
# minute=24  plantT=24  sensedT=25  H=0 C=1 RPS=4 ERR=0 extHeat=0
# [PASS] Cooler turned OFF below 25 C and fan returned OUT.
# [INFO] Injected sensor fault: +70 C
# [PASS] Safe-state outputs are correct at time 2426000
# [PASS] Main FSM entered M_SAFE for +70 C.
# [PASS] Sensor recovered from high fault.
# [INFO] Injected sensor fault: -20 C
# [PASS] Safe-state outputs are correct at time 2497000
# minute=25  plantT=24  sensedT=-20  H=0 C=0 RPS=0 ERR=1 extHeat=0
# [PASS] Main FSM entered M_SAFE for -20 C.
# ============================================================
# ALL AUTOMATIC CLOSED-LOOP TESTS PASSED
# ============================================================
# ** Note: $finish    : incubator_tb.v(249)
#    Time: 2587 ns  Iteration: 0  Instance: /incubator_tb
# 1
# Break in Module incubator_tb at incubator_tb.v line 249
# Simulation Breakpoint: 1
# Break in Module incubator_tb at incubator_tb.v line 249
# MACRO ./run.do PAUSED at line 11
```

شکل ۳: بخش پایانی رونوشت شامل تزریق خطاها و گزارش قبولی همه‌ی آزمون‌ها

ج) تفسیر مرحله‌به‌مرحله

با کنار هم گذاشتن جدول بالا، رونوشت و شکل موج، کلّ رفتار سامانه توضیح‌پذیر می‌شود:

- **راه‌اندازی سرد:** شبیه‌سازی از دمای 10 درجه آغاز می‌شود که کمتر از آستانهٔ 15 است؛ ازاین‌رو ماشین اصلی بی‌درنگ به حالت گرمایش می‌رود و گرم‌کننده روشن می‌شود. دما در هفت دقیقه با آهنگ سه درجه در دقیقه از 10 به 31 می‌رسد.

- **پایان گرمایش:** در دقیقهٔ هفتم دما به 31 می‌رسد که بزرگ‌تر از 30 است و گرم‌کننده خاموش می‌شود. توجّه شود که در دمای 25 یا 28، با آنکه محفظه دیگر سرد نیست، گرمایش ادامه یافته است؛ این همان پسماند خواستهٔ نمودار حالت است.

- **اعمال بار گرمایی**: پس از خاموشی گرم‌کننده، یک بار گرمایی بیرونی به اندازهٔ چهار درجه در دقیقه اعمال می‌شود و دما بالا می‌رود.

- **آستانه‌ی اکید در عمل**: در دقیقهٔ نهم دمای حس‌شده دقیقاً 35 است و خنک‌کننده هنوز روشن نمی‌شود، زیرا شرط دستورکار $T > 35$ است و نه $T \geq 35$. تنها در دقیقهٔ بعد و با دمای 39 سرمایش آغاز می‌گردد.

- **پلّکان افزایشی دور پنکه**: با شروع سرمایش، دور از چهار آغاز می‌شود؛ در دمای 42 به شش و در دمای 46 به هشت می‌رسد. هر سه سطح دور خواسته‌شده در دستورکار در همین بازه دیده می‌شوند.

- **پلّکان کاهشی دور پنکه**: پس از برداشتن بار بیرونی، دما به‌سرعت افت می‌کند. در دمای 40 دور همچنان هشت می‌ماند، چون شرط کاهش $T < 40$ است و نه $T \leq 40$. با رسیدن به 34 هر دو گذار کاهشی پیاپی رخ می‌دهند و دور مستقیم از هشت به چهار می‌رسد.

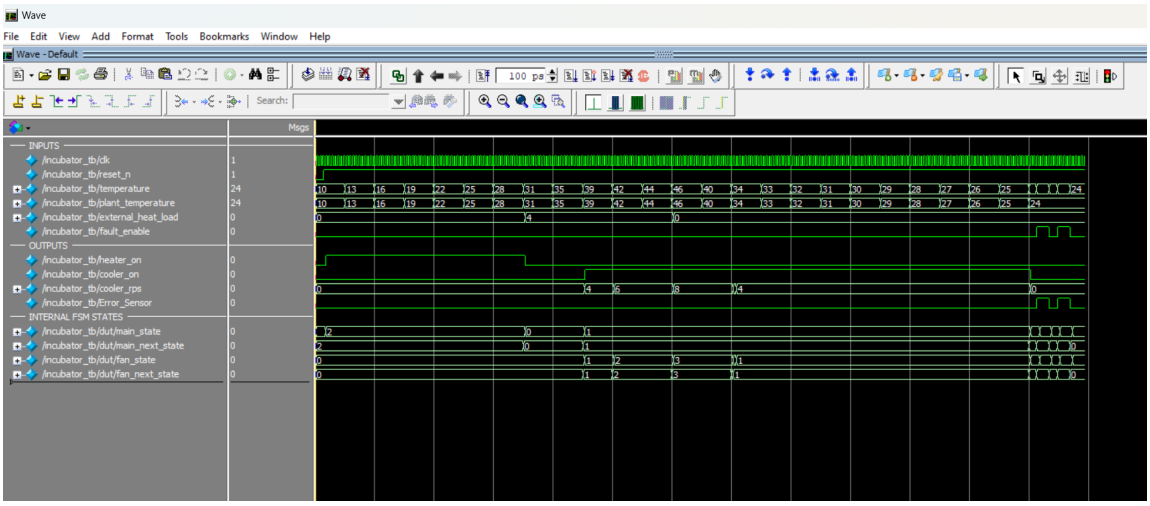
- **چرا دو گذار در یک دقیقه**: دورهٔ ساعت ۱۰ نانوثانیه و هر دقیقهٔ شبیه‌سازی ۱۰۰ نانوثانیه است؛ یعنی در هر دقیقه ده لبهٔ فعّال ساعت وجود دارد. ازاین‌رو ماشین‌ها فرصت دارند چند گذار پیاپی را پیش از رسیدن نمونهٔ بعدی حسگر انجام دهند و رونوشت، دو پیام کاهش دور را پشت سر هم چاپ می‌کند.

- **خاموشی خنک‌کننده**: از دقیقهٔ ۱۵ تا ۲۴ دما با آهنگ یک درجه در دقیقه پایین می‌آید تا در دقیقهٔ ۲۴ به 24 برسد که کمتر از 25 است. در همین لحظه خنک‌کننده خاموش و ماشین پنکه به حالت OUT بازمی‌گردد و دور صفر می‌شود؛ آزمون نیز صفر بودن دور را جداگانه وارسی می‌کند.

- **تزریق خطای بالا و پایین**: با جایگزینی مقدار حسگر با $+70$ و سپس -20، هر دو بیرون از بازهٔ معتبر، پرچم خطا یک و هر سه فرمان خروجی صفر می‌شوند و ماشین اصلی وارد حالت امن می‌گردد. پس از برداشتن خطا، پرچم پاک می‌شود و سامانه به کار عادی بازمی‌گردد.

- **پایان آزمون**: پس از هر دو تزریق، شمارندهٔ خطای میز آزمون صفر می‌ماند و پیام قبولی همهٔ آزمون‌ها چاپ می‌شود.

شکل موج کامل، همین دنباله را به‌صورت دیداری نشان می‌دهد: پلّکان صعودی دما در بازهٔ گرمایش، پرش پرچم گرم‌کننده و سپس خنک‌کننده، پلّکان دور پنکه با مقادیر چهار، شش و هشت، و در انتها دو پالس کوتاه پرچم خطا هم‌زمان با صفر شدن همهٔ فرمان‌ها.



شکل ۴: شکل موج کامل شبیه‌سازی حلقه‌بسته

۸.۶ سنتز، رمزگذاری حالت و تحلیل زمانی

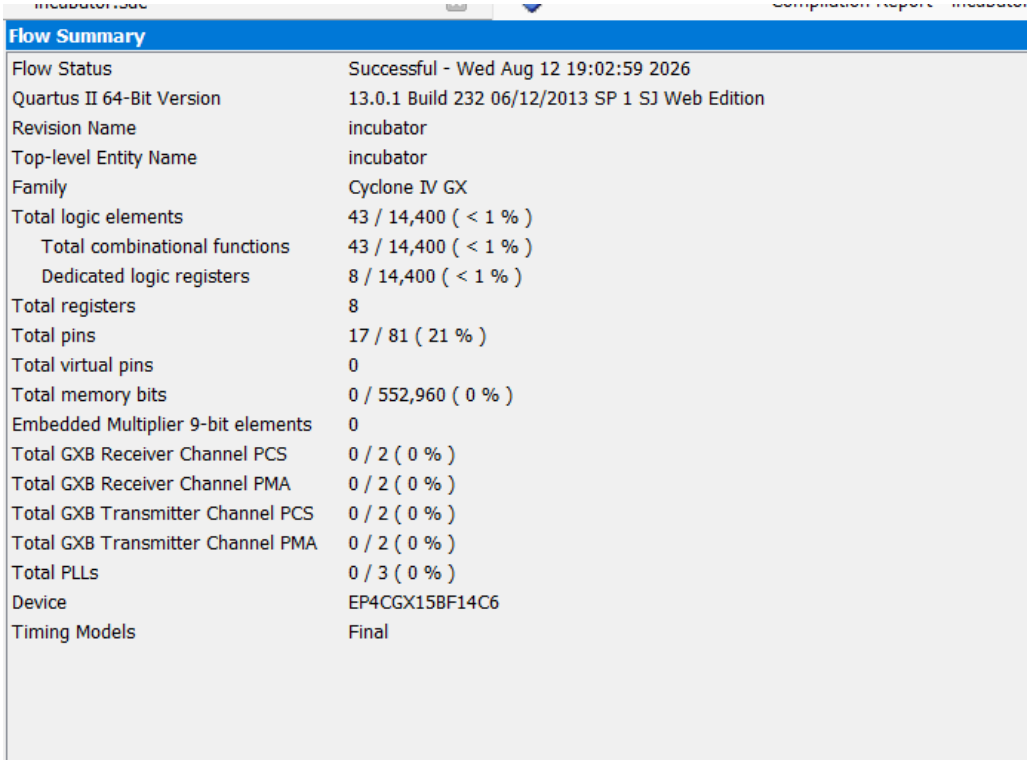
الف) مصرف منابع

واحد کنترل در Quartus II 13.0.1 و برای خانوادهٔ Cyclone IV GX همگردانی شد. انتخاب قطعه روی حالت خودکار گذاشته شد و ابزار قطعهٔ EP4CGX15BF14C6 را برگزید.

Flow Summary

Flow Status	Successful - Wed Aug 12 18:48:03 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	incubator
Top-level Entity Name	incubator
Family	Cyclone IV GX
Total logic elements	43
Total combinational functions	43
Dedicated logic registers	8
Total registers	8
Total pins	17
Total virtual pins	0
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0
Total GXB Receiver Channel PMA	0
Total GXB Transmitter Channel PCS	0
Total GXB Transmitter Channel PMA	0
Total PLLs	0

شکل ۵: خلاصه‌ی گزارش پس از تحلیل و سنتز



Flow Summary

Flow Status	Successful - Wed Aug 12 19:02:59 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	incubator
Top-level Entity Name	incubator
Family	Cyclone IV GX
Total logic elements	43 / 14,400 (< 1 %)
Total combinational functions	43 / 14,400 (< 1 %)
Dedicated logic registers	8 / 14,400 (< 1 %)
Total registers	8
Total pins	17 / 81 (21 %)
Total virtual pins	0
Total memory bits	0 / 552,960 (0 %)
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0 / 2 (0 %)
Total GXB Receiver Channel PMA	0 / 2 (0 %)
Total GXB Transmitter Channel PCS	0 / 2 (0 %)
Total GXB Transmitter Channel PMA	0 / 2 (0 %)
Total PLLs	0 / 3 (0 %)
Device	EP4CGX15BF14C6
Timing Models	Final

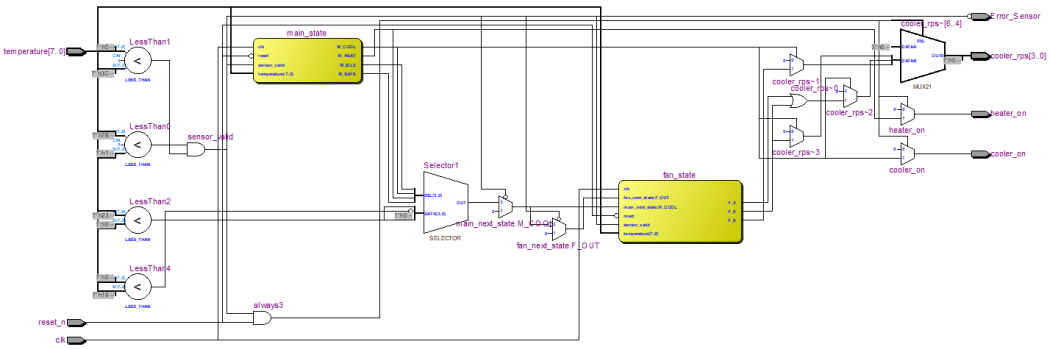
شکل ۶: خلاصه‌ی گزارش پس از همگردانی کامل

شمار پایه‌ها ۱۷ است و مستقیماً از روی واسط پیمانه قابل بازبینی است: ساعت، بازنشانی و هشت بیت دما در ورودی، و گرم‌کننده، خنک‌کننده، چهار بیت دور پنکه و پرچم خطا در خروجی. مصرف عناصر منطقی در همهٔ پیکربندی‌ها کمتر از یک درصد ظرفیّت قطعه است.

ب) نماهای سطح انتقال ثبات‌ها و ماشین حالت

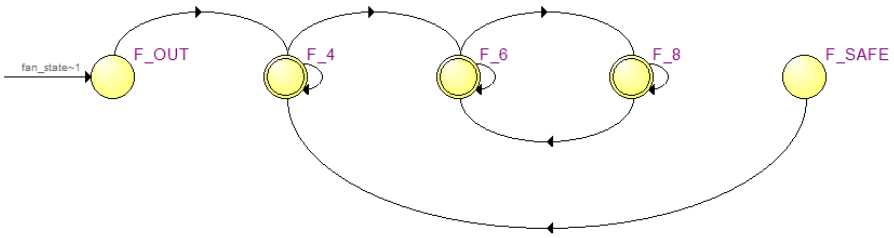
نمای سطح انتقال ثبات‌ها[۱۴] ساختار مورد انتظار را نشان می‌دهد: چهار مقایسه‌کنندهٔ آستانه در سمت ورودی، دروازهٔ تولید سیگنال اعتبار حسگر، دو بلوک ماشین حالت با برچسب‌های خوانا، و تسهیم‌کننده‌های منطق خروجی در سمت راست. مسیر وابستگی ماشین پنکه به حالت بعدی ماشین اصلی نیز به‌روشنی دیده می‌شود.

[14] Register Transfer Level (RTL) View



شکل ۷: نمای سطح انتقال ثبات‌های واحد کنترل

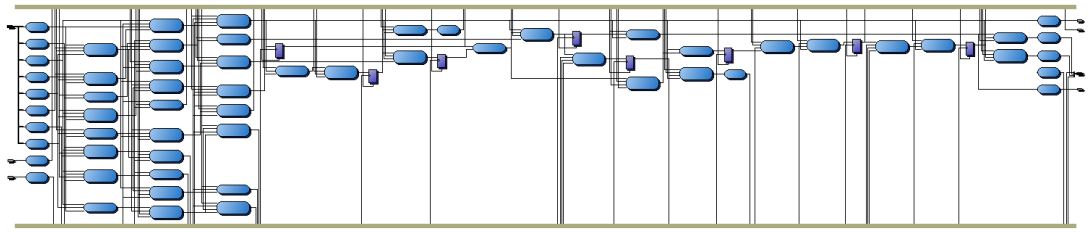
نمای ماشین حالت[۱۵] که ابزار مستقیماً از کد استخراج کرده، ماشین دور پنکه را با پنج حالت نشان می‌دهد و ساختار زنجیره‌ای گذارها میان دورهای چهار، شش و هشت در آن پیداست. نکتهٔ خواندنی این نما آن است که حالت امن پنکه یال ورودی ندارد؛ زیرا ورود به آن نه از درون ساختار انتخاب حالت، بلکه با بازنویسی مستقیم سیگنال اعتبار حسگر انجام می‌شود و ابزار استخراج ماشین حالت این مسیر را به‌صورت یال ترسیم نمی‌کند.



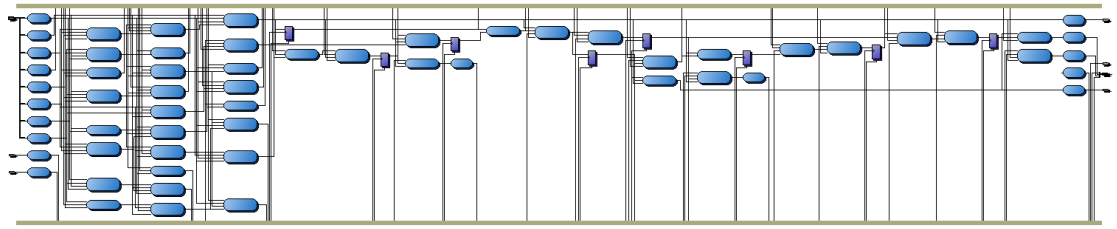
شکل ۸: نمای ماشین حالت دور پنکه استخراج‌شده از کد

نماهای فناوری[۱۶] پس از نگاشت و پس از جانمایی نیز نشان می‌دهند که این ساختار به جدول‌های جست‌وجو و فلیپ‌فلاپ‌های واقعی قطعه نگاشت و مسیریابی شده است.

[15] State Machine Viewer
[16] Technology Map Viewer



شکل ۹: نمای Technology Map Viewer (Post-Mapping) واحد کنترل



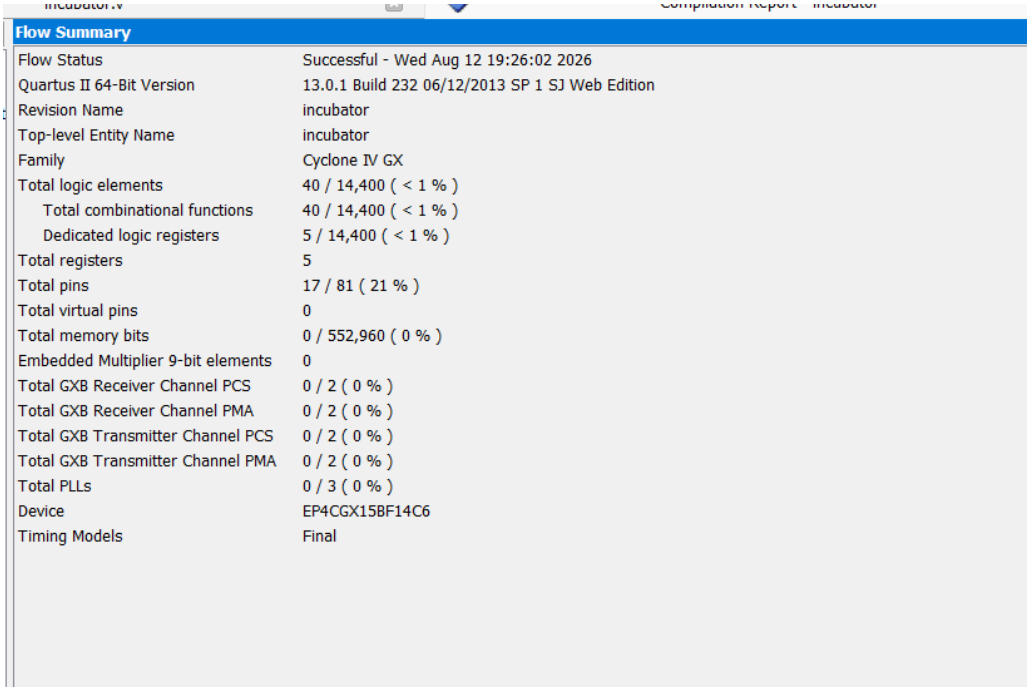
شکل ۱۰: نمای Technology Map Viewer (Post-Fitting) واحد کنترل

ج) مقایسه‌ی دو رمزگذاری حالت

چون کلّ طرح از دو ماشین حالت ساخته شده است، این آزمایش فرصت خوبی برای سنجش اثر رمزگذاری حالت بر نتیجهٔ سنتز فراهم می‌کند. طرح یک بار با رمزگذاری کمینه‌بیت[۱۷] و یک بار با رمزگذاری یک‌داغ[۱۸] همگردانی شد. تنها تفاوت دو پروژه، یک تنظیم سراسری در پروندهٔ پیکربندی است و کد توصیف سخت‌افزار در هر دو یکسان است.

[17] Minimal-Bits (Binary) Encoding

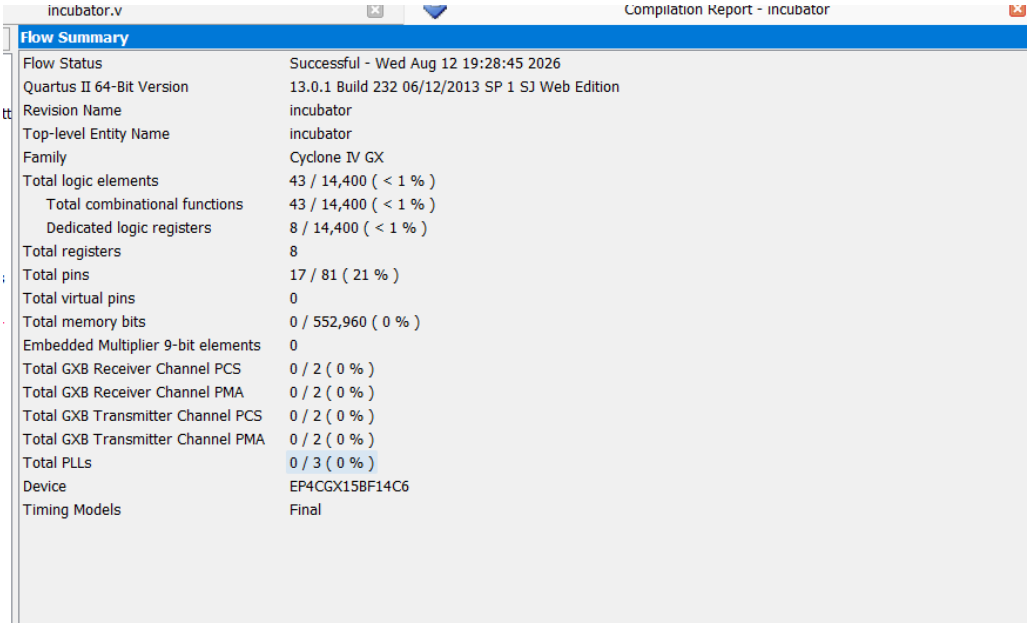
[18] One-Hot Encoding



Flow Summary

Flow Status	Successful - Wed Aug 12 19:26:02 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	incubator
Top-level Entity Name	incubator
Family	Cyclone IV GX
Total logic elements	40 / 14,400 (< 1 %)
Total combinational functions	40 / 14,400 (< 1 %)
Dedicated logic registers	5 / 14,400 (< 1 %)
Total registers	5
Total pins	17 / 81 (21 %)
Total virtual pins	0
Total memory bits	0 / 552,960 (0 %)
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0 / 2 (0 %)
Total GXB Receiver Channel PMA	0 / 2 (0 %)
Total GXB Transmitter Channel PCS	0 / 2 (0 %)
Total GXB Transmitter Channel PMA	0 / 2 (0 %)
Total PLLs	0 / 3 (0 %)
Device	EP4CGX15BF14C6
Timing Models	Final

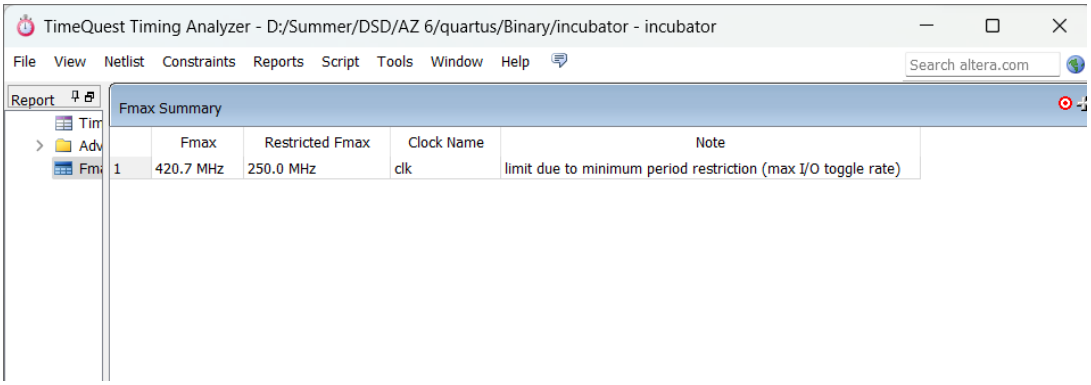
شکل ۱۱: خلاصه‌ی گزارش با رمزگذاری کمینه‌بیت



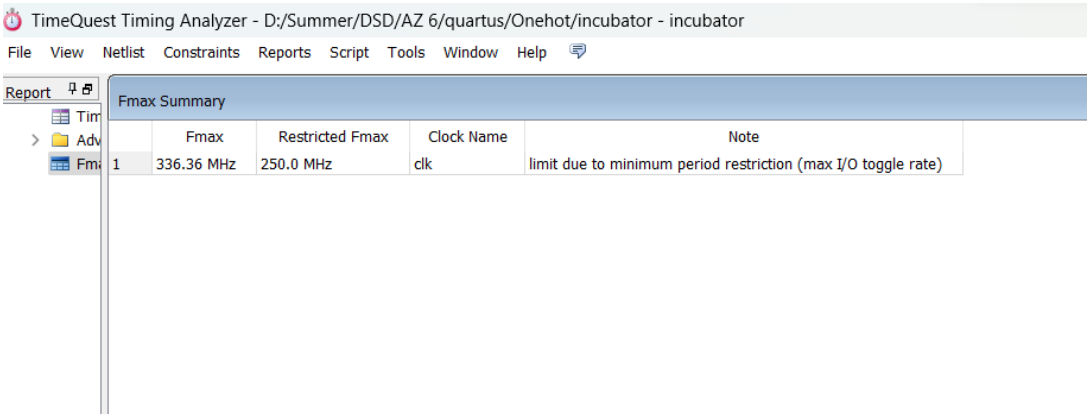
Flow Summary

Flow Status	Successful - Wed Aug 12 19:28:45 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	incubator
Top-level Entity Name	incubator
Family	Cyclone IV GX
Total logic elements	43 / 14,400 (< 1 %)
Total combinational functions	43 / 14,400 (< 1 %)
Dedicated logic registers	8 / 14,400 (< 1 %)
Total registers	8
Total pins	17 / 81 (21 %)
Total virtual pins	0
Total memory bits	0 / 552,960 (0 %)
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0 / 2 (0 %)
Total GXB Receiver Channel PMA	0 / 2 (0 %)
Total GXB Transmitter Channel PCS	0 / 2 (0 %)
Total GXB Transmitter Channel PMA	0 / 2 (0 %)
Total PLLs	0 / 3 (0 %)
Device	EP4CGX15BF14C6
Timing Models	Final

شکل ۱۲: خلاصه‌ی گزارش با رمزگذاری یک‌داغ



شکل ۱۳: بیشینه‌بسامد با رمزگذاری کمینه‌بیت



شکل ۱۴: بیشینه‌بسامد با رمزگذاری یک‌داغ

رمزگذاری	عناصر منطقی	توابع ترکیبی	ثبات	بیشینه‌بسامد
کمینه‌بیت	40	40	5	420.70 MHz
یک‌داغ	43	43	8	336.36 MHz

جدول ۶: مقایسه‌ی نتیجه‌ی سنتز برای دو رمزگذاری حالت

شمار ثبات‌ها در حالت کمینه‌بیت دقیقاً با انتظار نظری می‌خواند: ماشین اصلی چهار حالت دارد و به دو بیت، و ماشین پنکه پنج حالت دارد و به سه بیت نیاز دارد؛ در مجموع پنج ثبات. رمزگذاری یک‌داغ به‌جای آن هشت ثبات مصرف می‌کند، یعنی سه ثبات بیشتر.

نتیجهٔ درخور توجّه آنکه در این طرح، رمزگذاری یک‌داغ نه‌تنها مساحت بیشتری گرفته بلکه بیشینه‌بسامد را نیز کاهش داده است؛ این خلاف انتظار متعارفی است که یک‌داغ را به‌بهای مساحت، سریع‌تر می‌داند. توضیح آن در ماهیّت این ماشین‌ها نهفته است: مسیر بحرانی طرح، رمزگشایی حالت نیست بلکه زنجیرهٔ مقایسه‌کننده‌های هشت‌بیتی دماست که پیش از منطق حالت بعدی قرار دارد. با تعداد اندک حالت‌ها، رمزگشایی دوبیتی هزینهٔ چندانی ندارد، در حالی که یک‌داغ ثبات‌های بیشتری به مسیر می‌افزاید. بنابراین برتری هر رمزگذاری را باید برای هر طرح جداگانه سنجید و نمی‌توان آن را قاعده‌ای عمومی گرفت.

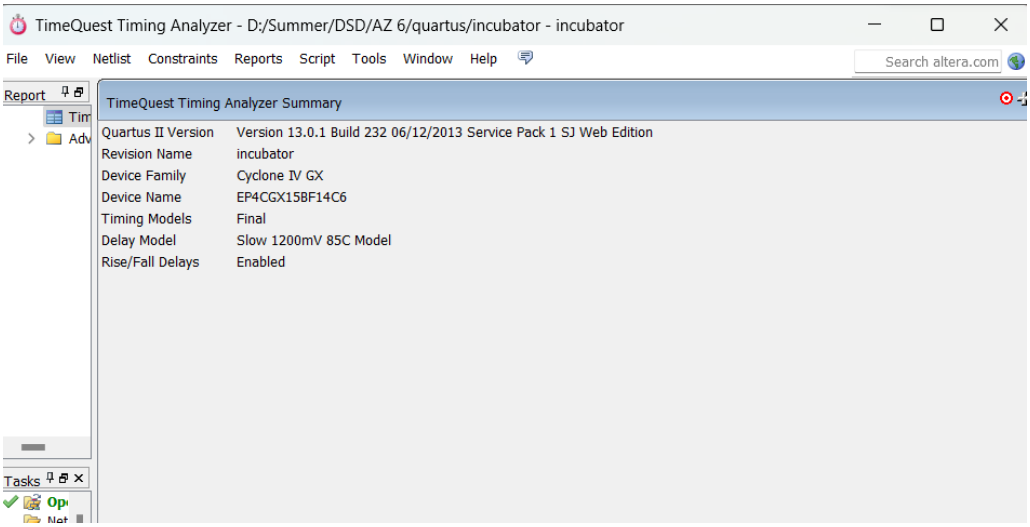
د) تحلیل زمانی

برای طرح یک پروندهٔ قیود طراحی[19] نوشته شد که ساعتی با دورهٔ ۲۰ نانوثانیه، معادل ۵۰ مگاهرتز، تعریف می‌کند.

```
create_clock -name clk -period 20.000 [get_ports {clk}]
```

کد ۸: پرونده‌ی قیود طراحی و تعریف ساعت

تحلیل‌گر زمانی[20] با مدل کند در دمای ۸۵ درجه و ولتاژ ۱۲۰۰ میلی‌ولت، یعنی بدترین شرایط کاری، اجرا شد.



شکل ۱۵: خلاصه‌ی تحلیل‌گر زمانی و مدل تأخیر به‌کاررفته

بیشینه‌بسامد[21] گزارش‌شده در بهترین پیکربندی 420.70 MHz است؛ یعنی بلندترین مسیر داده حدود 2.4 نانوثانیه طول می‌کشد و قید ۲۰ نانوثانیه‌ای با حاشیه‌ای در حدود ۱۷ نانوثانیه برآورده می‌شود. در هر دو پیکربندی، بیشینه‌بسامد مقیّد[22] برابر 250 MHz گزارش شده است و یادداشت خودِ گزارش علّت را روشن می‌کند: این حدّ از منطق درونی طرح نمی‌آید، بلکه از محدودیّت نرخ تغییر پایه‌های ورودی و خروجی[23] قطعه ناشی می‌شود.

شایان ذکر است که در کاربرد واقعی، حسگر تنها یک بار در دقیقه نمونه می‌دهد و بنابراین بسامد کاری واقعی سامانه هیچ‌گاه به این حدود نزدیک نمی‌شود. طرح نسبت به آهنگ نمونه‌برداری بی‌تفاوت است و در شبیه‌سازی، هر دقیقه با ۱۰۰ نانوثانیه مدل شده تا زمان اجرا معقول بماند.

[19] Synopsys Design Constraints
[20] TimeQuest Timing Analyzer
[21] Maximum Frequency (Fmax)
[22] Restricted Fmax
[23] Maximum I/O Toggle Rate

با توجّه به برگزاری مجازی آزمایشگاه، ارزیابی این آزمایش بر پایهٔ شبیه‌سازی، سنتز و تحلیل زمانی نرم‌افزاری انجام شده و در پرونده‌های پیکربندی پروژه تخصیص پایه‌ای برای کلیدها و چراغ‌های برد ثبت نشده است؛ ازاین‌رو ادّعایی دربارهٔ آزمون فیزیکی روی برد مطرح نمی‌شود.

۹.۶ نتایج

- هر دو نمودار حالت دستورکار، با همهٔ حالت‌ها و یال‌ها و با نامساوی‌های اکید، به‌صورت دو ماشین حالت مور پیاده‌سازی شدند.
- قاعدهٔ پیوند دو ماشین رعایت شد: ماشین دور پنکه تنها هنگام روشن بودن خنک‌کننده فعّال است و در غیر آن حالت به OUT بازمی‌گردد و دور صفر گزارش می‌شود.
- شبیه‌سازی حلقه‌بسته با مدل فرایند، چرخهٔ کامل گرمایش، توقّف، سرمایش با هر سه سطح دور، و بازگشت به حالت بی‌اثر را بدون هیچ خطا پیمود.
- آزمون هدفمند مرزها، برای هر آستانه هم مقدار برابر و هم نخستین مقدار پس از آن را سنجید و هر بیست و پنج وارسی آن با موفّقیّت گذشت.
- پسماند خواسته‌شده در نمودار حالت آزموده و تأیید شد؛ از جمله آنکه گرمایش پس از آغاز، تا عبور دما از ۳۰ درجه ادامه می‌یابد.
- پایش سلامت حسگر افزوده شد و در برابر مقادیر بیرون از بازهٔ معتبر، خروجی‌ها بی‌درنگ امن شدند و ماشین‌ها در لبهٔ بعدی وارد حالت امن گردیدند.
- سنتز با ۱۷ پایه و کمتر از یک درصد ظرفیّت منطقی قطعه انجام شد.
- مقایسهٔ دو رمزگذاری حالت نشان داد که برای این طرح، رمزگذاری کمینه‌بیت هم منابع کمتری می‌گیرد و هم بیشینه‌بسامد بالاتری می‌دهد.

نتیجه‌گیری

در این آزمایش واحد کنترل دیجیتال یک انکوباتور با دو ماشین حالت هم‌کار طراحی و ارزیابی شد. آنچه این طرح را از یک ماشین حالت ساده متمایز می‌کند، وابستگی شرطی دو ماشین به یکدیگر است؛ و دیدیم که استفاده از حالت بعدی ماشین اصلی به‌جای حالت جاری آن، یک پنجرهٔ ناسازگاری یک‌پالسی میان فرمان خنک‌کننده و دور پنکه را از میان می‌برد. دو میز آزمون مکمّل، دو پرسش متفاوت را پاسخ دادند: شبیه‌سازی حلقه‌بسته نشان داد سامانه در یک سناریوی واقع‌نما پایدار عمل می‌کند و به وضعیّت مطلوب

می‌رسد، و آزمون هدفمند مرزها نشان داد که تک‌تک آستانه‌ها، از جمله رفتار در دمای دقیقاً برابر با آستانه، مطابق نمودار حالت‌اند. سرانجام، مقایسهٔ دو رمزگذاری حالت یادآور شد که قواعد سرانگشتی طراحی، جای اندازه‌گیری را نمی‌گیرند: در این طرح که مسیر بحرانی آن را مقایسه‌کننده‌های دما تعیین می‌کنند، رمزگذاری یک‌داغ برخلاف انتظار رایج، هم کندتر و هم پرهزینه‌تر بود.